CHAPTER 21

Ruby Style

As you become familiar with Ruby and the Ruby community, you'll see references to "Ruby style" or "idiomatic Ruby." These terms refer to the way in which developers who really love Ruby tend to write Ruby code. Ruby is such a flexible language—demonstrated in the range of allowed syntax, in the different ways it has to do similar tasks, and in the ways it allows you to interact with existing code—that style and community standards of practice go a long way toward keeping Ruby code readable from project to project.

There are two distinct kinds of Ruby style. The first is the kind that is syntax-based, governs how you write individual lines and small blocks of code, and can (to some extent) be evaluated by a linting program. In this chapter, we'll discuss two tools that can check your code to see if it matches style rules: RuboCop and Standard. Often, these tools can automatically reformat your code to align with style guidelines.

The second kind of Ruby style is more nebulous, and it involves larger decisions than individual lines and has to do more with how to use Ruby as a tool. We'll discuss how to lean into Ruby's dynamism and support of dynamic typing (sometimes called "duck typing") as a way to make your code's intent clearer and give you less code to maintain and change in the future.

Before we say anything else about style in this chapter, it's important to remember that style is subjective, and nearly all of these guidelines are dependent on what you and your team find clear and readable. Team style is a case where consistency is more important than perfection—any time spent arguing over where the square brackets go is time lost. You want to avoid "bikeshedding"—spending inordinate time on trivial cosmetic issues like what color to paint the bike shed, rather than the important issues of your project. We'll offer some advice, but our overall recommendation is to pick the tooling to support a common set of practices and then spend your time on larger and more interesting code issues.

Written Ruby Style

The goal of having a written coding style is to make the intent and functionality of the code clear and easy to maintain. When the physical layout of the file matches its logical layout and the constructs are presented consistently, it's easier for a reader of the code to understand what the code is doing.

Programming languages tend to offer flexibility in how the code is actually laid out in the file as written by the developer. Some languages arrange their syntax in such a way as to be